

Geomatrix Detailed File Inventory and Technical Breakdown

Document Purpose

This document provides a detailed file-by-file understanding of the Geomatrix project. It is organized in page-style sections.

Page 1 — Project Overview and Scope

Project Title

Geomatrix

Objective

Geomatrix is a land-acquisition and infrastructure delay intelligence platform. It predicts risk, explains bottlenecks,

Core Stack

Frontend: Next.js with React and TypeScript
Backend: FastAPI with Python
Database: SQLite via SQLAlchemy
ML: scikit-learn, SHAP, numpy, pandas
AI Layer: Gemini for explanation and mitigation recommendations
Deployment: Vercel-friendly frontend with backend route proxying

Business Purpose

Detect project delay signals early
Rank at-risk projects by risk score
Explain the cause of risk using SHAP values
Recommend interventions using Gemini
Keep the UI stable and preserve existing structure

Safety Requirements Maintained

Only use real labeled data for model training
Reject synthetic and test-class records
Require a valid two-class dataset before training
Do not fabricate delay-day predictions without a verified regression target
Show honest status messages when the model is unavailable

Page 2 — Root Configuration and Setup Files

Root-level configuration

package.json: defines frontend dependencies and scripts
next.config.ts: controls Next.js behavior and API proxying
tsconfig.json: TypeScript build and alias configuration
eslint.config.mjs: linting rules
vercel.json: deployment configuration
next-env.d.ts: environment typings for Next.js

Documentation files

README.md: main project overview and setup
README_NEW.md: additional notes and updated instructions
tb_information_summary.txt: project status summary for TB/model review
all_prompts.txt: prompts and task history
scripts_seed.txt: seed or operational script notes

Purpose of the root layer

The root configuration is responsible for frontend package management, build scripts, deployment configuration, and documentation.

Page 3 — Frontend Shell and Shared UI Components

app/layout.tsx

Defines the outer HTML shell and global app wrapper. It is responsible for layout-level settings and app

app/globals.css

Provides the global styling architecture, theme variables, component spacing, color palette, cards, tables, and layout s

components/Shell.tsx

Main application shell containing the sidebar navigation, header, and global UI wrapper.

components/ExportDropdown.tsx

Dropdown control that allows users to export data to CSV, Excel, and PDF formats.

lib/data.ts

Contains core domain types, risk categories, project states, and static reference metadata.

lib/apiClient.ts

The type-safe API client for frontend-to-backend communication. It centralizes project, analytics, prediction, and alert

lib/exportUtils.ts

Handles export logic for report generation and file output.

lib/reports.ts

Formats report summaries, KPIs, and business text output.

Page 4 — Frontend Page Architecture Part I

app/page.tsx

Default entry page of the application.

app/login/page.tsx

Authentication page that supports Google login patterns and demo entry flows.

app/dashboard/page.tsx

Executive dashboard with KPIs, aggregated metrics, charts, and high-level portfolio visibility.

app/projects/page.tsx

Project register page showing project cards, filters, status values, and list-level project operations.

app/projects/[id]/page.tsx

Project detail page containing the predictive risk view, explainability panel, mitigation suggestions, and risk simulato

app/map/page.tsx

Map module showing spatial distribution of project locations using GIS data.

app/alerts/page.tsx

Alert center for active problems, delay signals, and follow-up actions.

Page 5 — Frontend Page Architecture Part II

app/data/page.tsx

Handles ingestion and upload for project and historical datasets.

app/model/page.tsx

Model monitoring and training UI. Displays the active model status, metrics, and training controls.

app/reports/page.tsx

Reporting interface for summary and export-oriented reporting.

app/analytics/page.tsx

Provides aggregates for project stages, risk drivers, and analytics comparisons.

app/settings/page.tsx

Contains configuration, preferences, and system setting controls.

app/admin/model/page.tsx

Administrative backend view for model management-related functionality.

app/api/

This folder contains API route handlers used by the Next.js app to proxy requests to the backend.

Important API routes

- app/api/gemini/explain/route.ts
- app/api/predict/route.ts
- app/api/projects/route.ts
- app/api/projects/[id]/...
- app/api/analytics/overview/route.ts
- app/api/analytics/drivers/route.ts
- app/api/analytics/stages/route.ts
- app/api/map/geojson/...
- app/api/alerts/route.ts
- app/api/reports/route.ts

Page 6 — Backend Core and Database Layer

geomatrix_v2/main.py

Main FastAPI app entry point. Registers the routers and initializes the database at startup.

geomatrix_v2/database.py

Defines SQLAlchemy engine and SQLite session management. Includes migration logic to safely add columns if missing.

geomatrix_v2/models.py

Contains the schema definitions for the application's core entities.

Key database tables

- Project
- HistoricalDelayRecord
- ModelRun
- RiskPrediction
- Alert
- DataIngestionLog
- GeminiInsight

Project model role

The Project table stores project metadata, fields such as land area, affected families, legal issues, compensation statu

HistoricalDelayRecord role

This table stores training records with real delay outcomes used to build the model.

ModelRun role

Stores metrics, model version, algorithm, timestamps, and the path to serialized model artifact files.

Page 7 — Backend Schemas and Data Validation

geomatrix_v2/schemas.py

Defines typed request/response models using Pydantic.

Why this matters

Schemas validate:

- project creation payloads
- project update payloads
- prediction outputs
- training outputs
- ingestion result metadata
- dashboard and alerts structures

Example categories in the schema layer

- ProjectOut
- ProjectCreate
- ProjectUpdate
- PredictionOut
- TrainResponse
- IngestResult
- IngestionLogOut

Data integrity principle

The schema layer supports validation but the real safety enforcement happens in the ML pipeline and ingestion logic, esp

Page 8 — Ingestion and Data Import Layer

geomatrix_v2/routers/ingest.py

This is the ingestion router that accepts project and historical CSV/JSON uploads.

What it does

- reads uploaded files
- normalizes values
- maps aliases such as land_required_ha, project_id, delay_flag, and source_record_id
- validates required fields
- stores records in database tables
- marks data_classification as REAL or TEST depending on the source name

Data validation rules

- missing delayed labels are rejected
- missing required historical fields are skipped
- duplicate source_record_id entries are ignored
- explicit boolean labels are enforced for delayed values

Risk of fake data

This layer is intentionally careful about classification tags so that test and synthetic records are not treated as trus

Page 9 — ML Feature Engineering and Training Pipeline

geomatrix_v2/ml/features.py

Contains the canonical feature schema used across training and predictions.

Canonical feature columns

- land_area_ha
- affected_families
- pending_claims
- legal_cases
- doc_completeness_pct
- approval_pending
- rr_pending
- overdue_milestones
- compensation_pending
- env_clearance_pending

Why this is important

These are the same fields used for both training and prediction. This prevents drift between the feature vector used in

geomatrix_v2/ml/train.py

This module trains the model from real records.

Training safety behavior

- minimum record threshold is enforced
- one-class datasets are rejected

synthetic/test records are filtered out
the model is only considered valid if the data is real and two-class
model metadata includes confidence, class distribution, and real-label provenance

Model output

It saves serialized artifacts and an active model reference file.

Page 10 — ML Prediction, Explainability, and Evaluation

geomatrix_v2/ml/predict.py

Loads the active model and computes risk for a new project.

Prediction contract

If no valid model exists, return honest status:
Model not trained — insufficient real labeled data.
Delay probability is bounded between 0 and 1
Confidence is constrained to the valid range
Predicted delay days are not fabricated without a valid regression target

geomatrix_v2/ml/explain.py

Generates SHAP-based feature explanations for the active model.

geomatrix_v2/ml/evaluate.py

Calculates performance metrics such as precision, recall, F1, ROC-AUC, and confusion-based summaries.

Why this matters

Model predictions must be honest, explainable, and auditable. The app should never pretend a model exists when it is abs

Page 11 — API Routers and Business Logic Layer

geomatrix_v2/routers/projects.py

Responsible for project CRUD and project-level risk routes.

geomatrix_v2/routers/ml.py

Handles model training and model status operations.

geomatrix_v2/routers/alerts.py

Returns alert lists and acknowledges alerts after review.

geomatrix_v2/routers/map.py

Creates GeoJSON for the GIS view.

geomatrix_v2/routers/gemini.py

Builds prompts from project information and queries Gemini for mitigation recommendations.

geomatrix_v2/routers/auth.py

Handles authentication and user identification relevant to the app layer.

Role of the router layer

The routers are the integration point between the UI, the database, the ML layer, and the AI recommendation layer.

Page 12 — Test Suite and Verification

geomatrix_v2/tests/test_ml_pipeline.py

Covers model safety tests such as:
insufficient labeled data rejection
one-class training rejection
confidence range checks
missing-model prediction behavior
SHAP availability rules

geomatrix_v2/tests/test_real_pipeline_api.py

Covers end-to-end behavior including:

- CSV upload and ingestion
- training from real data
- prediction generation
- SHAP persistence
- validation of required historical labels

Verification command

The project root pytest suite was run successfully:

- 16 passed
- exit code 0

This proves the application is stable under the project-level regression suite and that the honesty constraints continue

Page 13 — Data and Artifact Storage

geomatrix_v2/sample_data/

Stores reference or sample project data used during development.

geomatrix_v2/uploads/

Stores file uploads created during ingestion workflows.

geomatrix_v2/static/

Contains legacy static assets and frontend resources.

geomatrix_v2/model_artifacts/

Holds trained model files, metrics metadata, and the pointer to the active model run.

Important operational note

The active model file must be checked against metadata before it is trusted. Stale or invalid artifacts should be reject

Page 14 — Final Project Summary

What the repository does well

- Keeps the UI and workflow intact
- Uses real project data and real delay labels
- Supports risk prediction and explainability
- Supports Gemini-based recommendations as a supporting layer only
- Maintains honest model status behavior under uncertainty

What the pipeline emphasizes

- Data provenance
- Real supervised learning
- Guardrails against synthetic or invalid data
- Honest no-model behavior
- Feature consistency across training, prediction, and explanation

Final conclusion

Geomatrix is a production-minded risk intelligence platform designed around transparency, explainability, and correctness